Credit Card Transaction Fraud Detection

Supervised Fraud Model: Final Project Report

Prepared by: Hanwei Xu

436 Fraud Analysis

November 2th 2026

Table of Contents

1. Executive Summary 3
2. Description of the Data 4
3. Data Cleaning 6
4. Variable Creation 8
5. Feature Selection 10
6. Preliminary Model Exploration 12
7. Final Model Performance 15
8. Financial Curves and Recommended Cutoff 19
9. Summary 21

Appendix: Data Quality Report 22

1. Executive Summary

Card fraud is a direct cost to the business: every fraudulent purchase that goes through is money lost, and every legitimate purchase that is wrongly stopped costs revenue and customer goodwill. We were asked to build a fraud score that ranks every credit card purchase by how likely it is to be fraud, so the business can stop or review the riskiest transactions before the loss occurs. I used one year (2010) of about 98,000 real government purchase-card transactions, of which 2,283 were confirmed frauds. From this history we built a large set of signals that describe how a card is normally used, such as how often it is used, where, and for how much, and how a new purchase differs from that pattern. I kept the 20 most useful signals and tested several machine learning methods. The final model is a neural network.

To make sure the results will hold up in real use, the model was judged on the last two months of the year (November and December), which were never used to build it. On these months, **reviewing just the riskiest 3% of transactions catches 60.6% of all fraud**. Assuming each fraud caught saves $500, each legitimate transaction stopped costs $40, and a portfolio of 10 million transactions per year, I recommend setting the score cutoff so that **the top 4% of transactions are flagged**. At this cutoff the model catches about two thirds of all fraud and is expected to **save about $60 million per year** after the cost of false alarms, close to the maximum possible savings, while only about 1.3 legitimate transactions are stopped for every fraud caught.

2. Description of the Data

The data set is **Card Transactions Data**: credit card purchase transactions made by employees of a U.S. government organization over the 2010 calendar year, with labeled fraud records. It has **10 fields and 98,161 records**. Each record is one transaction with a record number, card number, date, merchant number, merchant description, merchant state and zip, transaction type, amount and the fraud label. There are 2,283 frauds, an overall fraud rate of 2.33%. The full data quality report (DQR) is in the Appendix; its summary tables and three of the most important field distributions are shown here.

Table 2.1. Numeric fields

Field	# Records with Values	% Populated	# Zeros	Min	Max	Mean	Std. Dev.	Most Common
Date	98,161	100.00%	0	2010-01-01	2010-12-31	—	—	2010-02-28
Amount	98,161	100.00%	0	$0.01	$3,102,045.53	$424.82	$9,934.14	$3.62

Date is a datetime field, so the mean and standard deviation are not reported.

Table 2.2. Categorical fields

Field	# Records with Values	% Populated	# Zeros	# Unique Values	Most Common
Recnum	98,161	100.00%	0	98,161	n/a (all unique)
Cardnum	98,161	100.00%	0	1,645	5142148452
Merchnum	94,752	96.53%	237	13,091	930090121224
Merch description	98,161	100.00%	0	13,126	GSA-FSS-ADV
Merch state	96,955	98.77%	0	227	TN
Merch zip	93,439	95.19%	0	4,567	38118
Transtype	98,161	100.00%	0	4	P
Fraud	98,161	100.00%	95,878	2	0

The 237 zeros in Merchnum are the text value "0", a placeholder for a missing merchant number. Recnum, Cardnum and Merch zip look like numbers but are identifiers, so they are treated as categorical.

Important field distributions

Amount. Amounts range from $0.01 to $3.1 million, with a median of $137.20; the distribution is highly right-skewed. The count drops by more than an order of magnitude just above $2,500, which suggests a single-purchase limit on these cards. This finding motivated the split-purchase variables in Section 4. The $3.1 million transaction is a clear outlier (see Section 3).

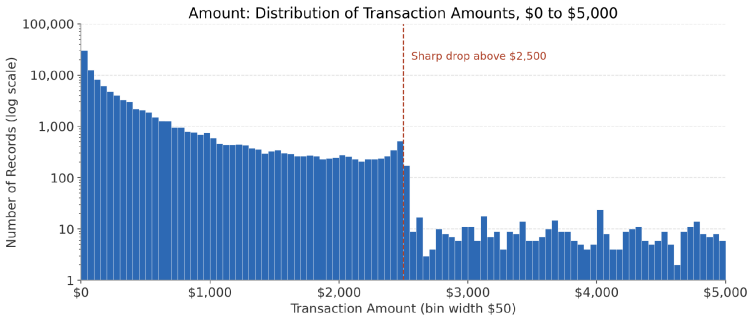


Figure 2.1. Distribution of transaction amounts, $0 to $5,000 (log scale; 99.67% of records).

Date. All 365 days of 2010 are present. Daily volume follows a strong weekly cycle (6 to 684 transactions per day, average 269). Because the data are ordered in time, the last two months (November and December) are held out as out-of-time validation data.

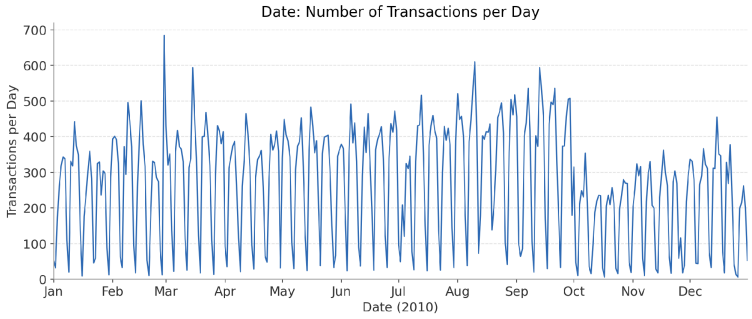


Figure 2.2. Number of transactions per day in 2010.

Merchnum. There are 13,091 merchant numbers. The most common merchant (930090121224, 9.6% of all records) is FedEx shipping charges, including many small repeated charges of $3.62 to $3.80. Very common merchants like this behave very differently from the long tail of merchants a card rarely uses.

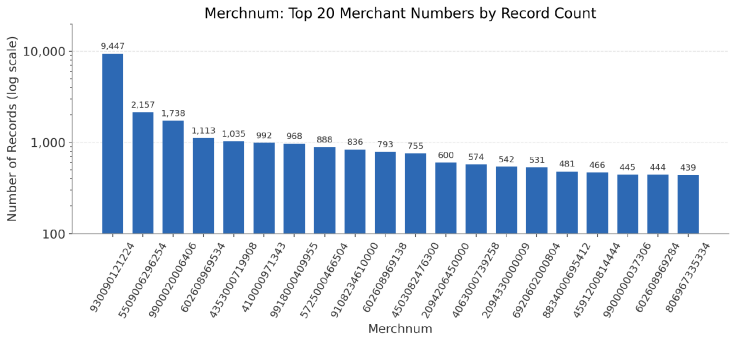


Figure 2.3. Top 20 merchant numbers by number of transactions (log scale).

3. Data Cleaning

What and why. Before building variables, the data must contain only the transactions we want to score, must not be distorted by data errors, and must have a value in every key field so that every transaction can be linked to its card and merchant history. Cleaning therefore has three steps: exclusions, outlier treatment and imputation of missing values. No fraud records are removed.

Table 3.1. Record counts through cleaning

Step	Records Removed	Records Remaining	Frauds Remaining
Raw data file	—	98,161	2,283
Exclude Transtype other than P	355	97,806	2,283
Remove $3.1 million outlier	1	97,805	2,283

3.1 Exclusions

Only purchase transactions are kept. Transtype has four values: P (97,806), A (181), D (173) and Y (1). P most likely means purchase, while the other codes are probably authorizations, declines or other non-purchase events. The fraud model is meant to score purchases, so the 355 non-P records are excluded; none of them are frauds. This also removes 180 of the 186 records with corrupted three-digit state codes. The 7,592 FedEx charges of $3.62 to $3.80 are kept, but they are flagged and left out of the Benford's law variables, where their repeated digits would distort the result.

3.2 Outliers

One transaction of $3,102,045.53 (record 53314, INTERMEXICO, no merchant number, state or zip) is 65 times larger than the next largest amount ($47,900). It is not a fraud, and it would distort every amount-based variable for its card, so it is removed. No other outliers are removed: in a fraud problem, unusually large amounts are often exactly what we want to find.

3.3 Imputation

Merchnum, Merch state and Merch zip have missing values. Each missing value is filled with the most appropriate value for that record, using related fields in the same record, in the order Merchnum → state → zip so that each imputed field can help fill the next.

Table 3.2. Imputation logic

Field	# Missing	How missing values are filled (in order)
Merchnum	3,291	1) Look up the merchant number from the same merchant description (1,166 filled). 2) Retail credit/debit adjustments set to "unknown" (697). 3) Each remaining description (515 of them, 1,428 records) gets a new unique merchant number.
Merch state	1,031	1) Zip-to-state lookup, with 22 zips added by hand (74). 2) Merchnum and description lookups (2). 3) Adjustments set to "unknown" (658). 4) Non-U.S. codes (Canadian provinces, numeric codes) grouped as "foreign" (203). 5) Remaining records set to "unknown" (297).
Merch zip	4,366	1) Merchnum-to-zip lookup (1,686). 2) Description-to-zip lookup (45). 3) Adjustments set to "unknown" (688). 4) Known U.S. state: most populous zip in that state (1,414). 5) Remaining records set to "unknown" (533). Leading zeros lost in the raw file are restored.

After cleaning, every one of the 97,805 records has a value in all fields. A review of the cleaning code also found possible improvements (using the most common value in each lookup table, using the full zip code database for zip-to-state, and setting adjustments to "unknown" only when the value is missing); together these would reduce the number of "unknown" zip codes from 533 to 321.

4. Variable Creation

Why. The raw fields say little about fraud on their own. Fraud shows up as a change in behavior compared with an entity's own history, so the variables describe each transaction in the context of earlier transactions. Once a card number is stolen, it is often used many times in a short period (a burst of activity), at merchants, states and zip codes the card has never used, for amounts that are unusual for that card, and sometimes split into several purchases to stay under the $2,500 limit or repeated with the same amount to test the card. Fraud can also be tied to a merchant, so the same ideas are applied to merchants and to combinations of fields.

How. Variables are built for 23 **entities**: single fields (card, merchant) and combinations such as card + merchant, card + zip, card + state, card + day of week and merchant + description. For each entity, the variables summarize all earlier transactions in look-back windows of 0 (same day), 1, 3, 7, 14, 30 and 60 days. Every variable uses only the current and earlier records, so there is no look-ahead. Target-encoded variables are fit on the training period only (before November 1).

Table 4.1. Candidate variables created

Variable group	Description	# Variables
Target encoding	Average fraud rate of merchant state, zip and day of week (training data only).	4
Benford's law	How far the first digits of a card's or merchant's amounts depart from Benford's law.	2
Days since	Days since the previous transaction with the same entity.	23
Velocity (frequency)	Number of transactions with the entity in the last {0, 1, 3, 7, 14, 30, 60} days.	161
Amount statistics	Average, maximum, median and total amount with the entity in each window.	644
Actual / statistic ratios	Current amount divided by each amount statistic above.	644
Relative velocity	Short-window count or amount (0, 1 day) divided by long-window value (7–60 days).	368
Velocity / days-since	Relative count velocity divided by (days since + 1).	184
Amount variability	Mean, max and median of (earlier amount − current amount) in each window.	414
Unique counts	Number of distinct values of entity B seen with entity A in each window (e.g., distinct merchants per card in 7 days).	696
Squared relative velocity	Relative count velocity divided by window length squared.	184
Geography	Miles from the card's previous merchant zip; flag if over 1,000 miles.	2
Amount category and flags	Amount quintile; foreign, weekend, whole-dollar amount, high amount for merchant, merchant dominance, state change within a day.	7
Amount vs. own history [new]	Z-score and percentile of the amount vs. the card's or merchant's last 30/60 days.	8
Split purchases [new]	Card (or card at merchant) daily total over $2,500 while each purchase is ≤ $2,500; near-limit counts.	4
Repeated amounts [new]	Earlier transactions with exactly the same amount for the card, merchant or card at merchant.	6
Card–merchant loyalty [new]	Share of the card's earlier transactions at this merchant.	1
Total created		3,352

Variable group	Description	# Variables
After removing duplicate columns	Exact duplicate columns (from duplicate entities and constant variables) removed.	2,663

The 19 new variables ([new]) were added to the class notebook, and seven existing variables (for example Amount_Rounded and state_inconsistency_flag) were corrected for index and data-type errors. The split-purchase flag is a strong single signal: transactions where it is 1 have a 19.6% fraud rate, compared with 1.5% for the rest.

5. Feature Selection

What and why. 2,663 candidate variables are far too many for a model: most are weak or highly correlated with each other, and a model with too many inputs is slow, hard to explain and likely to overfit. The goal is a small set of about 20 variables that together carry almost all of the fraud signal.

How. Selection uses a filter followed by a wrapper, on the records from January 15 to October 31 (81,729 records, 1,828 frauds). The first two weeks are left out because the look-back variables are not yet well formed, and November–December are kept for out-of-time testing.

- **Filter:** each variable is ranked by its univariate Kolmogorov–Smirnov (KS) statistic between frauds and non-frauds. A random variable ranked 2,526 of 2,664 (KS 0.019), so the top of the list is real signal. The top 1,000 variables are kept.
- **Wrapper:** forward sequential feature selection on the 1,000 filtered variables. At each step it adds the variable that most increases the fraud detection rate at 3% (FDR@3%) of a small LGBM model (20 trees, 4 leaves), using 2-fold cross-validation, until 20 variables are selected.

Nine wrapper runs were compared, changing one setting at a time: the size of the filter pool (50 to 1,000), the wrapper model (LGBM, random forest, CatBoost) and the direction (forward or backward). A bigger filter pool helped the most (FDR@3% with 20 variables rose from 0.691 with 50 variables to 0.735 with 1,000), because it lets the wrapper find variables that add new information even when their own KS is low. Backward selection was worse and slower, and the random forest wrapper was noisy.

Table 5.1. Wrapper runs (20 variables, 2-fold CV, FDR@3%)

Run	Direction	Wrapper model	Filter pool	FDR@3% (20 vars)	Saturates at
1	Forward	LGBM	50	0.691	10 vars
2	Forward	LGBM	200	0.702	6 vars
3	Forward	LGBM	500	0.715	6 vars
4	**Forward**	**LGBM**	**1,000**	**0.735**	**9 vars**
5	Forward	Random forest (seed 1)	200	0.717	20 vars
6	Forward	Random forest (seed 2)	200	0.706	17 vars
7	Forward	CatBoost	200	0.718	8 vars
8	Backward	LGBM	50	0.679	8 vars
9	Backward	LGBM	100	0.689	10 vars

Highlighted row = final choice (Run 4).

Result. Run 4 (forward, LGBM, pool of 1,000) gives the best performance, 0.735, with the smallest fold-to-fold spread. Its performance curve rises quickly and is flat after about 9–10 variables, so 20 variables is about twice the saturation point.

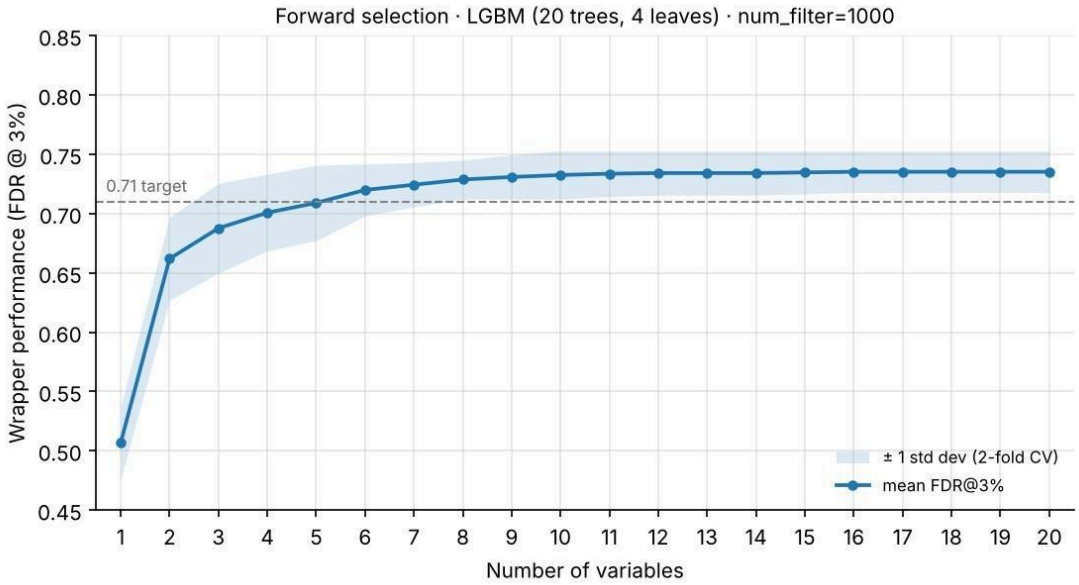


Figure 5.1. Model performance vs. number of variables for the final wrapper run (band = ±1 std. dev. across CV folds).

Table 5.2. Final 20 variables, in wrapper order, with the univariate filter measure (KS)

Order	Variable	Univariate KS	Used in final model
1	Cardnum_unique_count_for_card_state_1	0.509	Yes
2	Card_Merchdesc_State_total_7	0.286	Yes
3	Card_dow_count_0_by_60	0.359	Yes
4	card_zip_actual/toal_60	0.166	Yes
5	Amount_Rounded	0.226	Yes
6	card_state_count_1	0.255	Yes
7	Card_Merchdesc_State_max_30	0.241	Yes
8	Cardnum_count_7	0.531	Yes
9	Cardnum_total_amount_1_by_7	0.271	Yes
10	Cardnum_actual/med_7	0.159	Yes
11	Cardnum_amount_zscore_60	0.167	
12	Merchnum_desc_max_60	0.210	
13	Cardnum_unique_count_for_card_state_7	0.470	
14	Card_dow_variability_max_7	0.413	
15	card_state_count_3	0.230	
16	Card_Merchdesc_Zip_max_30	0.241	
17	Card_dow_count_30	0.398	
18	Card_dow_variability_max_14	0.366	
19	Card_dow_count_0_by_60_sq	0.359	
20	Card_dow_actual/max_7	0.357	

The final neural network uses the first 10 variables (Section 6). The 20 variables cover 7 entities and every time window from the same day to 60 days.

6. Preliminary Model Exploration

Setup. The data are split by time. Records from January 1 to October 31 (85,189 records, 1,948 frauds) are the modeling data; each model run randomly splits them into 70% training and 30% testing. November 1 to December 31 (12,616 records, 335 frauds) is the out-of-time (OOT) validation set, which measures how a model will perform on future data. The split is set by date (the first November record), not by a fixed row number. The 20 variables are z-scaled, capped at ±10 standard deviations and rescaled. Performance is measured by FDR@3%, the share of all frauds found in the 3% of records with the highest scores. Because each run uses a different random split, every hyperparameter setting is run 10 times and the results are averaged.

Algorithms explored. One linear baseline and five nonlinear models were tuned:

- **Logistic regression (baseline):** a weighted sum of the inputs passed through an S-shaped (logistic) function to give a probability. Simple and stable, but it can only draw a linear boundary. Best (default settings, 10 variables): 0.697 test, 0.485 OOT.
- **Decision tree:** splits the data with a sequence of yes/no questions on single variables; the fraud rate in each final leaf is the score. Easy to explain but unstable and prone to overfitting. Best (depth 8, entropy, min leaf 30): 0.735 test, 0.554 OOT.
- **Random forest:** averages many deep trees, each built on a bootstrap sample of the records and a random subset of variables, which reduces the variance of a single tree. Best (60 trees, depth 12, entropy): 0.781 test, 0.514 OOT.
- **Boosted trees (LightGBM and CatBoost):** build many small trees in sequence, each one fitted to the errors of the trees before it. Usually the strongest method for tabular data. Best LGBM (500 trees, 8 leaves, learning rate 0.02): 0.805 test, 0.565 OOT. Best CatBoost (300 iterations, depth 4): 0.794 test, 0.554 OOT.
- **Neural network (multi-layer perceptron):** inputs feed a hidden layer of nodes, each a weighted sum passed through a nonlinear activation, which feed the output score. It learns smooth nonlinear relationships. Best (one hidden layer of 20 tanh nodes, alpha 0.01): 0.746 test, 0.600 OOT.

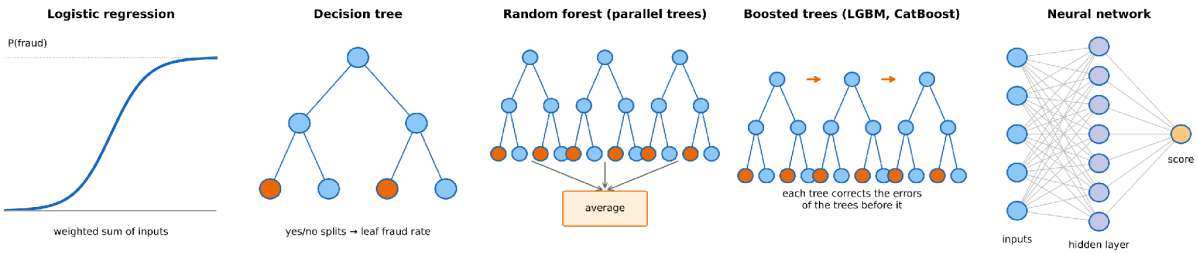


Figure 6.1. How each algorithm turns the input variables into a fraud score.

Hyperparameter exploration. Tables 6.1 and 6.2 show the settings tried for each model (mean of 10 runs). Very bad models (training − testing gap over 0.08 or OOT under 0.47) are left out. Complexity was increased step by step (depth, number of trees, leaves, nodes) while watching the gap between training and testing, which shows overfitting.

Run	Model	# vars	Hyperparameters	Trn	Tst	OOT	Trn–Tst
1	Logistic regression	10	default	0.694	0.697	0.485	-0.002
2	Logistic regression	20	default	0.697	0.688	0.487	0.009
3	Decision tree	10	depth=5	0.702	0.688	0.489	0.014
4	Decision tree	10	depth=7	0.747	0.700	0.528	0.047
5	Decision tree	10	depth=10, min_split=100, min_leaf=50	0.749	0.719	0.533	0.030
6	Decision tree	10	depth=12, min_split=120, min_leaf=60	0.751	0.722	0.504	0.030
7	Decision tree	10	depth=15, min_split=200, min_leaf=100	0.729	0.702	0.507	0.027
8	Decision tree	10	depth=None, min_split=200, min_leaf=100	0.729	0.702	0.509	0.026
9	**Decision tree**	**10**	**depth=8, min_split=60, min_leaf=30, crit=entropy**	**0.767**	**0.735**	**0.554**	**0.032**
10	Decision tree	20	depth=12, min_split=120, min_leaf=60	0.762	0.739	0.512	0.023
11	Decision tree	10	depth=8, min_split=100, min_leaf=50, crit=entropy	0.754	0.730	0.544	0.024
12	Decision tree	10	depth=10, min_split=80, min_leaf=40, crit=entropy	0.774	0.732	0.529	0.042
13	Decision tree	20	depth=8, min_split=60, min_leaf=30, crit=entropy	0.786	0.738	0.515	0.049
14	Decision tree	20	depth=10, min_split=100, min_leaf=50, crit=entropy	0.783	0.748	0.501	0.035
15	Random forest	10	trees=20, depth=5	0.719	0.715	0.502	0.004
16	Random forest	10	trees=50, depth=8	0.793	0.740	0.508	0.053
17	Random forest	10	trees=50, depth=12, min_split=60, min_leaf=30	0.784	0.752	0.505	0.032
18	Random forest	10	trees=60, depth=15, min_split=80, min_leaf=40	0.777	0.750	0.506	0.026
19	Random forest	10	trees=60, depth=20, min_split=100, min_leaf=50, max_feat=3	0.767	0.744	0.508	0.022
20	Random forest	10	trees=100, depth=15, min_split=60, min_leaf=30, max_feat=3	0.793	0.762	0.503	0.030
21	Random forest	10	trees=60, depth=15, min_split=120, min_leaf=60, max_feat=5	0.763	0.746	0.507	0.017
22	Random forest	20	trees=60, depth=15, min_split=80, min_leaf=40	0.785	0.759	0.517	0.026
23	Random forest	10	trees=60, depth=10, min_split=40, min_leaf=20, crit=entropy	0.801	0.767	0.504	0.035
24	Random forest	10	trees=60, depth=12, min_split=60, min_leaf=30, crit=entropy, max_feat=5	0.807	0.770	0.510	0.037
25	**Random forest**	**20**	**trees=60, depth=12, min_split=60, min_leaf=30, crit=entropy**	**0.814**	**0.781**	**0.514**	**0.033**
26	Random forest	20	trees=100, depth=12, min_split=40, min_leaf=20, crit=entropy, max_feat=6	0.844	0.795	0.515	0.049

Table 6.1. Model exploration: logistic regression, decision tree, random forest (FDR@3%, mean of 10 runs; highlighted = best setting).

Run	Model	# vars	Hyperparameters	Trn	Tst	OOT	Trn–Tst
27	Boosted tree (LGBM)	10	trees=20, leaves=4	0.716	0.711	0.500	0.005
28	Boosted tree (LGBM)	10	trees=100, leaves=4	0.757	0.748	0.531	0.009
29	Boosted tree (LGBM)	10	trees=200, leaves=4, lr=0.05	0.760	0.745	0.522	0.015
30	Boosted tree (LGBM)	10	trees=300, leaves=6, lr=0.03	0.792	0.773	0.529	0.018
31	Boosted tree (LGBM)	10	trees=300, leaves=8, lr=0.03, min_child=50	0.807	0.775	0.538	0.033
32	Boosted tree (LGBM)	10	trees=500, leaves=8, lr=0.02, min_child=100, subsample=0.8, colsample=0.8	0.812	0.777	0.541	0.036
33	Boosted tree (LGBM)	20	trees=300, leaves=8, lr=0.03, min_child=50	0.841	0.800	0.555	0.040
34	**Boosted tree (LGBM)**	**20**	**trees=500, leaves=8, lr=0.02, min_child=100, subsample=0.8, colsample=0.8**	**0.847**	**0.805**	**0.565**	**0.042**
35	Boosted tree (LGBM)	20	trees=300, leaves=12, lr=0.03, min_child=100, lambda=5	0.847	0.807	0.556	0.040
36	Boosted tree (CatBoost)	10	iters=100, depth=3, lr=0.08, l2=5	0.736	0.717	0.527	0.019
37	Boosted tree (CatBoost)	10	iters=300, depth=4, lr=0.05, l2=5	0.793	0.772	0.564	0.022
38	**Boosted tree (CatBoost)**	**20**	**iters=300, depth=4, lr=0.05, l2=5**	**0.808**	**0.794**	**0.554**	**0.014**
39	Neural net	10	layers=(3)	0.710	0.707	0.507	0.003
40	Neural net	10	layers=(10)	0.730	0.724	0.545	0.006
41	Neural net	10	layers=(20), alpha=0.001	0.745	0.732	0.589	0.013
42	Neural net	10	layers=(30), alpha=0.01	0.748	0.737	0.583	0.011
43	Neural net	10	layers=(20,10), alpha=0.01	0.762	0.751	0.589	0.011
44	Neural net	10	layers=(20), alpha=0.01, lr_init=0.003, max_iter=300	0.756	0.735	0.586	0.021
45	**Neural net**	**10**	**layers=(20), act=tanh, alpha=0.01**	**0.749**	**0.746**	**0.600**	**0.002**
46	Neural net	10	layers=(20,10), alpha=0.01, early_stop=True	0.730	0.729	0.542	0.000
47	Neural net	10	layers=(20,10), act=tanh, alpha=0.01	0.774	0.759	0.586	0.015
48	Neural net	10	layers=(20), act=tanh, alpha=0.1	0.728	0.727	0.522	0.002
49	Neural net	20	layers=(20), act=tanh, alpha=0.01	0.800	0.764	0.589	0.036

Table 6.2. Model exploration: boosted trees and neural network (FDR@3%, mean of 10 runs; highlighted = best setting).

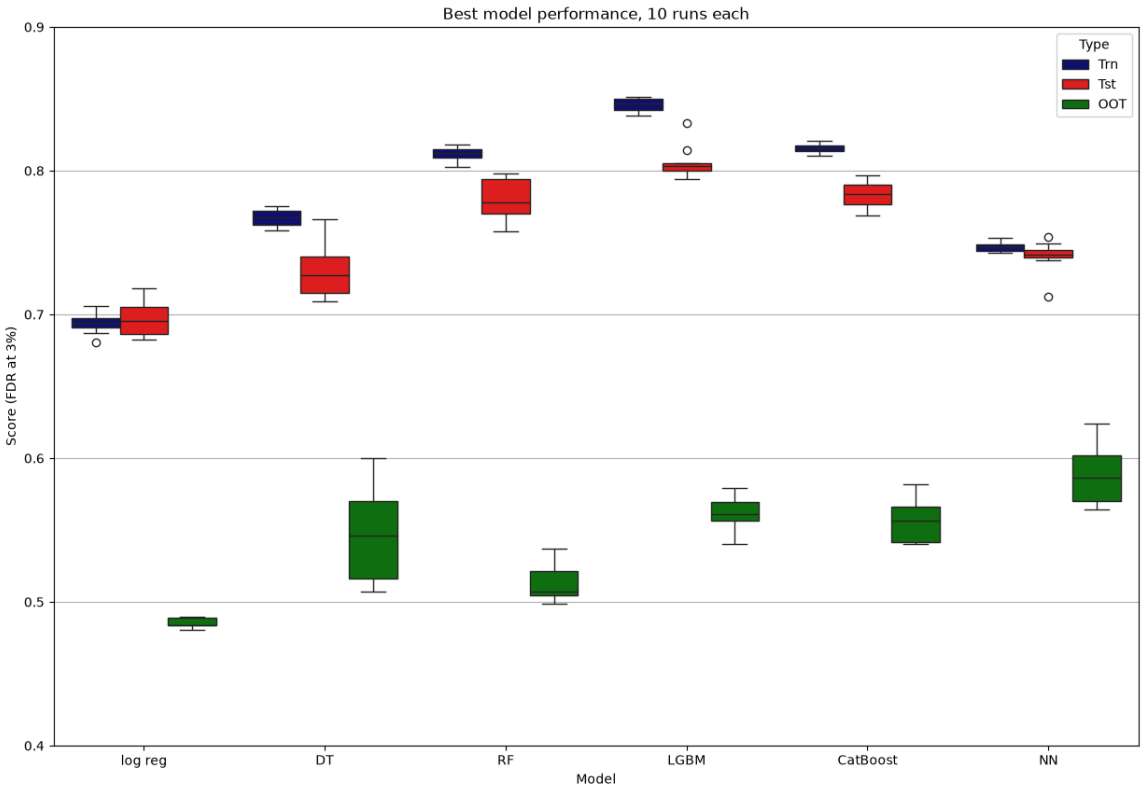


Figure 6.2. FDR@3% on training, testing and OOT data for the best setting of each model (10 runs each).

Observations. All nonlinear models beat logistic regression. The tree ensembles score highest on training and testing data but lose much more on the OOT months, and their training–testing gap is larger. The neural network has a lower test score than LGBM, but it has almost no gap between training and testing (0.749 vs. 0.746) and the **best OOT performance (0.600)**, so it generalizes best to new months. Since the OOT result is what the business can expect after implementation, the neural network is chosen as the final model. Decision trees and the neural network did best with the top 10 variables; the ensembles did best with all 20.

7. Final Model Performance

The final model is a **neural network (scikit-learn MLPClassifier)** with one hidden layer of 20 nodes, using the top 10 variables from Table 5.2. Table 7.1 lists the full model specification; every setting not listed there is the scikit-learn default.

Table 7.1. Final model specification

Item	Setting
Algorithm	MLPClassifier (multi-layer perceptron), scikit-learn
Input variables	Top 10 variables in wrapper order (Table 5.2)
Input scaling	z-scale, cap at ±10 standard deviations, z-scale again
hidden_layer_sizes (non-default)	(20,): one hidden layer with 20 nodes (default is (100,))
activation (non-default)	tanh (default is relu)
alpha (non-default)	0.01: L2 regularization (default is 0.0001)
random_state (non-default)	25: used for both the 70/30 train/test split and weight initialization
Other hyperparameters (defaults)	solver = adam, learning_rate_init = 0.001, batch_size = auto (200), max_iter = 200, early_stopping = False
Training data	70% random sample of Jan 1 – Oct 31 (59,632 records, 1,385 frauds); converged after 71 iterations
Testing / OOT data	30% of Jan 1 – Oct 31 (25,557 records, 563 frauds) / Nov 1 – Dec 31 (12,616 records, 335 frauds)

Choosing the final run. With the hyperparameters frozen, the model was trained 40 times with different random splits. The averages were 0.749 training, 0.742 testing and 0.598 OOT, with a standard deviation of only 0.014 on OOT, so the model is stable. The run with random_state = 25 was selected as the final model because its results are typical of the 40 runs and its training–testing gap is small:

	Training	Testing	OOT
FDR@3%, final model	**74.9%**	**74.6%**	**60.6%**
FDR@3%, mean of 40 runs	74.9%	74.2%	59.8%

Tables 7.2 to 7.4 show the score distribution of the final model in population bins of 1%, sorted from the highest score down. Only the top 20 bins are shown, since the decision point is in the first few percent. The left half describes each bin; the right half is cumulative up to and including that bin, so the % Bads column is the fraud detection rate at that cutoff. KS is % Bads − % Goods (cumulative), and FPR is the number of goods flagged for each bad. The 3% row is highlighted.

Table 7.2. Training results

Training	# Records	# Goods	# Bads	Fraud Rate	
	59,632	58,247	1,385	0.0232	

	Bin Statistics					Cumulative Statistics						
Pop. Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumul. Goods	Cumul. Bads	% Goods	% Bads (FDR)	KS	FPR
1	596	52	544	8.72%	91.28%	596	52	544	0.09%	39.28%	39.19	0.10
2	597	227	370	38.02%	61.98%	1,193	279	914	0.48%	65.99%	65.51	0.31
3	596	472	124	79.19%	20.81%	1,789	751	1,038	1.29%	**74.95%**	73.66	0.72
4	596	528	68	88.59%	11.41%	2,385	1,279	1,106	2.20%	79.86%	77.66	1.16
5	597	567	30	94.97%	5.03%	2,982	1,846	1,136	3.17%	82.02%	78.85	1.63
6	596	569	27	95.47%	4.53%	3,578	2,415	1,163	4.15%	83.97%	79.82	2.08
7	596	578	18	96.98%	3.02%	4,174	2,993	1,181	5.14%	85.27%	80.13	2.53
8	597	582	15	97.49%	2.51%	4,771	3,575	1,196	6.14%	86.35%	80.22	2.99
9	596	583	13	97.82%	2.18%	5,367	4,158	1,209	7.14%	87.29%	80.15	3.44
10	596	584	12	97.99%	2.01%	5,963	4,742	1,221	8.14%	88.16%	80.02	3.88
11	597	587	10	98.32%	1.68%	6,560	5,329	1,231	9.15%	88.88%	79.73	4.33
12	596	583	13	97.82%	2.18%	7,156	5,912	1,244	10.15%	89.82%	79.67	4.75
13	596	584	12	97.99%	2.01%	7,752	6,496	1,256	11.15%	90.69%	79.53	5.17
14	596	590	6	98.99%	1.01%	8,348	7,086	1,262	12.17%	91.12%	78.95	5.61
15	597	591	6	98.99%	1.01%	8,945	7,677	1,268	13.18%	91.55%	78.37	6.05
16	596	590	6	98.99%	1.01%	9,541	8,267	1,274	14.19%	91.99%	77.79	6.49
17	596	584	12	97.99%	2.01%	10,137	8,851	1,286	15.20%	92.85%	77.66	6.88
18	597	584	13	97.82%	2.18%	10,734	9,435	1,299	16.20%	93.79%	77.59	7.26
19	596	588	8	98.66%	1.34%	11,330	10,023	1,307	17.21%	94.37%	77.16	7.67
20	596	592	4	99.33%	0.67%	11,926	10,615	1,311	18.22%	94.66%	76.43	8.10

Table 7.3. Testing results

Testing	# Records	# Goods	# Bads	Fraud Rate	
	25,557	24,994	563	0.0220	

	Bin Statistics					Cumulative Statistics						
Pop. Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumul. Goods	Cumul. Bads	% Goods	% Bads (FDR)	KS	FPR
1	256	20	236	7.81%	92.19%	256	20	236	0.08%	41.92%	41.84	0.08
2	255	112	143	43.92%	56.08%	511	132	379	0.53%	67.32%	66.79	0.35
3	256	215	41	83.98%	16.02%	767	347	420	1.39%	**74.60%**	73.21	0.83
4	255	226	29	88.63%	11.37%	1,022	573	449	2.29%	79.75%	77.46	1.28
5	256	242	14	94.53%	5.47%	1,278	815	463	3.26%	82.24%	78.98	1.76
6	255	241	14	94.51%	5.49%	1,533	1,056	477	4.23%	84.72%	80.50	2.21
7	256	254	2	99.22%	0.78%	1,789	1,310	479	5.24%	85.08%	79.84	2.73
8	256	251	5	98.05%	1.95%	2,045	1,561	484	6.25%	85.97%	79.72	3.23
9	255	249	6	97.65%	2.35%	2,300	1,810	490	7.24%	87.03%	79.79	3.69
10	256	250	6	97.66%	2.34%	2,556	2,060	496	8.24%	88.10%	79.86	4.15
11	255	250	5	98.04%	1.96%	2,811	2,310	501	9.24%	88.99%	79.75	4.61
12	256	251	5	98.05%	1.95%	3,067	2,561	506	10.25%	89.88%	79.63	5.06
13	255	250	5	98.04%	1.96%	3,322	2,811	511	11.25%	90.76%	79.52	5.50
14	256	249	7	97.27%	2.73%	3,578	3,060	518	12.24%	92.01%	79.76	5.91
15	256	252	4	98.44%	1.56%	3,834	3,312	522	13.25%	92.72%	79.47	6.34
16	255	254	1	99.61%	0.39%	4,089	3,566	523	14.27%	92.90%	78.63	6.82
17	256	254	2	99.22%	0.78%	4,345	3,820	525	15.28%	93.25%	77.97	7.28
18	255	254	1	99.61%	0.39%	4,600	4,074	526	16.30%	93.43%	77.13	7.75
19	256	255	1	99.61%	0.39%	4,856	4,329	527	17.32%	93.61%	76.29	8.21
20	255	254	1	99.61%	0.39%	5,111	4,583	528	18.34%	93.78%	75.45	8.68

Table 7.4. Out-of-time (OOT) validation results

OOT	# Records	# Goods	# Bads	Fraud Rate	
	12,616	12,281	335	0.0266	

	Bin Statistics					Cumulative Statistics						
Pop. Bin %	# Records	# Goods	# Bads	% Goods	% Bads	Total # Records	Cumul. Goods	Cumul. Bads	% Goods	% Bads (FDR)	KS	FPR
1	126	24	102	19.05%	80.95%	126	24	102	0.20%	30.45%	30.25	0.24
2	126	65	61	51.59%	48.41%	252	89	163	0.72%	48.66%	47.93	0.55
3	126	86	40	68.25%	31.75%	378	175	203	1.42%	**60.60%**	59.17	0.86
4	127	107	20	84.25%	15.75%	505	282	223	2.30%	66.57%	64.27	1.26
5	126	119	7	94.44%	5.56%	631	401	230	3.27%	68.66%	65.39	1.74
6	126	119	7	94.44%	5.56%	757	520	237	4.23%	70.75%	66.51	2.19
7	126	124	2	98.41%	1.59%	883	644	239	5.24%	71.34%	66.10	2.69
8	126	119	7	94.44%	5.56%	1,009	763	246	6.21%	73.43%	67.22	3.10
9	126	119	7	94.44%	5.56%	1,135	882	253	7.18%	75.52%	68.34	3.49
10	127	126	1	99.21%	0.79%	1,262	1,008	254	8.21%	75.82%	67.61	3.97
11	126	121	5	96.03%	3.97%	1,388	1,129	259	9.19%	77.31%	68.12	4.36
12	126	124	2	98.41%	1.59%	1,514	1,253	261	10.20%	77.91%	67.71	4.80
13	126	119	7	94.44%	5.56%	1,640	1,372	268	11.17%	80.00%	68.83	5.12
14	126	125	1	99.21%	0.79%	1,766	1,497	269	12.19%	80.30%	68.11	5.57
15	126	121	5	96.03%	3.97%	1,892	1,618	274	13.17%	81.79%	68.62	5.91
16	127	127	0	100.00%	0.00%	2,019	1,745	274	14.21%	81.79%	67.58	6.37
17	126	126	0	100.00%	0.00%	2,145	1,871	274	15.23%	81.79%	66.56	6.83
18	126	123	3	97.62%	2.38%	2,271	1,994	277	16.24%	82.69%	66.45	7.20
19	126	125	1	99.21%	0.79%	2,397	2,119	278	17.25%	82.99%	65.73	7.62
20	126	123	3	97.62%	2.38%	2,523	2,242	281	18.26%	83.88%	65.62	7.98

Training and testing results are nearly identical (FDR@3% of 74.9% and 74.6%), confirming the model is not overfit. The OOT detection rate is lower (60.6%), as expected for future months whose fraud patterns have drifted from the training period; this is the performance to expect after implementation.

8. Financial Curves and Recommended Cutoff

The score cutoff decides how many transactions are flagged. A lower cutoff catches more fraud but also stops more legitimate purchases. To pick a cutoff, the OOT results are turned into expected annual dollars with these assumptions:

- Each fraud caught saves **$500** (green curve, fraud $ caught).
- Each legitimate transaction flagged (false positive) costs **$40** in lost revenue and customer friction (red curve).
- Overall savings = fraud $ caught − lost revenue (blue curve).
- The data are a sample of 100,000 records from a portfolio of 10 million transactions per year, and the OOT period is 2 of 12 months, so OOT dollars are multiplied by (12/2) × (10,000,000/100,000) = 600.

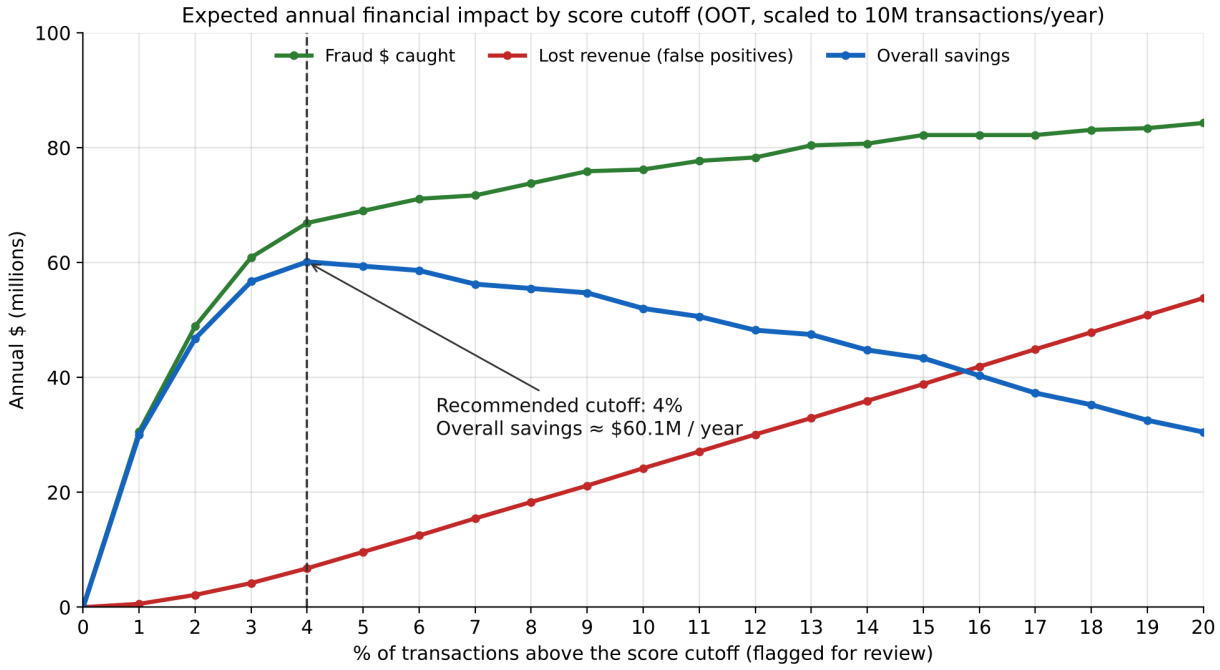


Figure 8.1. Fraud $ caught, lost revenue and overall savings per year by score cutoff (OOT data).

Table 8.1. Expected annual dollars at selected cutoffs (OOT)

Cutoff (% flagged)	Fraud caught (FDR)	Goods flagged per fraud	Fraud $ caught	Lost revenue	Overall savings
1%	30.4%	0.24	$30.6M	$0.6M	$30.0M
2%	48.7%	0.55	$48.9M	$2.1M	$46.8M
3%	60.6%	0.86	$60.9M	$4.2M	$56.7M
4%	**66.6%**	**1.26**	**$66.9M**	**$6.8M**	**$60.1M**
5%	68.7%	1.74	$69.0M	$9.6M	$59.4M
6%	70.7%	2.19	$71.1M	$12.5M	$58.6M
8%	73.4%	3.10	$73.8M	$18.3M	$55.5M
10%	75.8%	3.97	$76.2M	$24.2M	$52.0M

Recommendation: set the score cutoff at the top 4% of transactions. Overall savings rise steeply up to 3–4% and then flatten and slowly decline, because beyond this point each extra percent of flagged transactions adds few frauds but a steadily growing number of false positives. The 4% cutoff is at the left edge of the flat region: it gives the highest expected savings, **about $60.1 million per year** (fraud caught $66.9M − lost revenue $6.8M), catches 66.6% of fraud, and stops only about 1.3 legitimate transactions per fraud caught. Going to 5% or 6% adds no savings and stops about 40% to 85% more good customers; a tighter 3% cutoff would reduce review workload by a quarter but give up about $3.4 million per year.

9. Summary

We built a supervised fraud model for credit card purchases using 2010 transaction data from a U.S. government organization (98,161 records, 10 fields, 2,283 frauds). A data quality report described every field. Cleaning kept only purchase transactions (removing 355 non-purchase records), removed one $3.1 million outlier, and filled missing merchant numbers, states and zip codes with record-level logic based on related fields, leaving 97,805 records with no frauds lost.

Based on how card fraud happens (bursts of activity, new merchants and locations, unusual amounts, split and repeated purchases), we created 3,352 candidate variables over 23 entities and look-back windows of 0 to 60 days, including 19 new variables and 7 corrected ones. After removing duplicates, 2,663 candidates remained. A KS filter reduced these to 1,000, and a forward wrapper with an LGBM model, scored by FDR@3%, selected the final 20 variables; performance saturates at about 10 variables.

We then explored logistic regression, a decision tree, a random forest, LightGBM, CatBoost and a neural network, tuning each with 10 random 70/30 training/testing splits and checking each against out-of-time data from November–December. Boosted trees had the highest testing score, but the neural network (one hidden layer of 20 tanh nodes, alpha 0.01, top 10 variables) had almost no overfitting and the best OOT performance, so it was chosen. The final model has FDR@3% of 74.9% on training, 74.6% on testing and **60.6% on OOT**. Using $500 per fraud caught, $40 per false positive and 10 million transactions per year, we recommend flagging the top 4% of scores, with **expected savings of about $60 million per year**.

Further work. With more time we could: (1) select hyperparameters by repeated cross-validation, then retrain the frozen model on all training and testing data before final OOT validation; (2) try downsampling the non-frauds or upweighting frauds (e.g., SMOTE) during training; (3) cap long-tailed variables more tightly and remove the redundant _sq variable; (4) combine the neural network and LGBM scores in an ensemble, or segment the model by card type or merchant type; (5) build new variables aimed at the November–December drift, for example merchant-level fraud dynamics; and (6) calibrate the score so that score values map to fraud odds, and monitor performance after implementation with periodic rebuilds.

Appendix: Data Quality Report

A.1 Data Description

The data set is **Card Transactions Data**, which contains credit card purchase transactions made by employees of a U.S. government organization. The data came from real business purchases made over the 2010 calendar year, with labeled fraud records added. There are **10 fields and 98,161 records**.

A.2 Summary Tables

Table A.1. Numeric fields

Field	# Records with Values	% Populated	# Zeros	Min	Max	Mean	Std. Dev.	Most Common
Date	98,161	100.00%	0	2010-01-01	2010-12-31	—	—	2010-02-28
Amount	98,161	100.00%	0	$0.01	$3,102,045.53	$424.82	$9,934.14	$3.62

Date is a datetime field, so the mean and standard deviation are not reported.

Table A.2. Categorical fields

Field	# Records with Values	% Populated	# Zeros	# Unique Values	Most Common
Recnum	98,161	100.00%	0	98,161	n/a (all unique)
Cardnum	98,161	100.00%	0	1,645	5142148452
Merchnum	94,752	96.53%	237	13,091	930090121224
Merch description	98,161	100.00%	0	13,126	GSA-FSS-ADV
Merch state	96,955	98.77%	0	227	TN
Merch zip	93,439	95.19%	0	4,567	38118
Transtype	98,161	100.00%	0	4	P
Fraud	98,161	100.00%	95,878	2	0

The 237 zeros in Merchnum are the text value "0", a placeholder for a missing merchant number. Recnum, Cardnum and Merch zip look like numbers but are identifiers, so they are treated as categorical.

A.3 Visualization of Each Field

Field: Recnum

Description: Record number. This is an ordinal, unique positive integer for each transaction record, running from 1 to 98,161. Because every value is unique, no distribution is shown.

Field: Cardnum

Description: Credit card number used for the transaction. There are 1,645 unique cards, and all card numbers are 10 digits that begin with 51421 or 51422. The chart shows the 20 cards with the most transactions. The most active card, 5142148452, has 1,192 transactions.

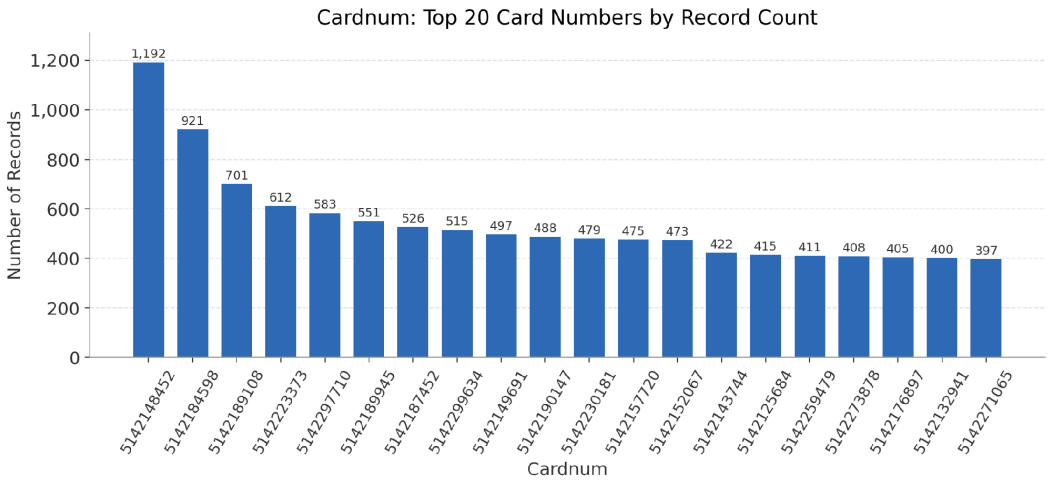


Figure A.1. Top 20 card numbers by number of transactions.

Field: Date

Description: Date of the transaction. All 365 days of 2010 are present. The first chart shows the number of transactions per day. Daily volume follows a strong weekly cycle, ranging from 6 to 684 transactions per day with an average of 269. The busiest day is February 28, 2010, with 684 transactions. The second chart shows transactions per full week. Volume rises gradually through the summer, peaks in late September, and drops sharply in mid-October, then stays near 1,000 to 1,900 per week for the rest of the year. The partial weeks at the start (Jan 1–3) and end (Dec 27–31) of the year are omitted from the weekly chart.

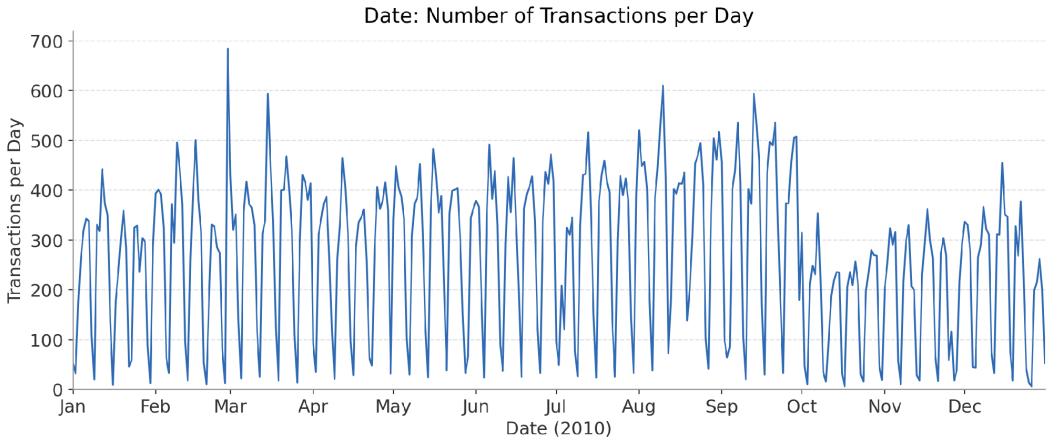


Figure A.2. Transactions per day.

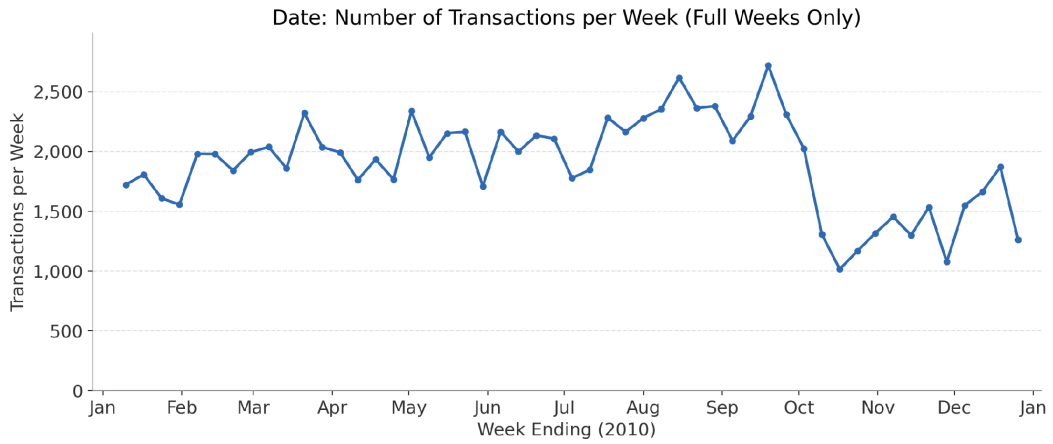


Figure A.3. Transactions per week.

Field: Merchnum

Description: Merchant number of the business where the transaction occurred. The field is 96.53% populated; 3,409 records are blank and a further 237 records contain the placeholder value "0". There are 13,091 unique merchant numbers. The chart shows the top 20 merchants on a log scale. The most common, 930090121224, accounts for 9,447 records (9.6% of the file); its merchant descriptions are FedEx shipping charges.

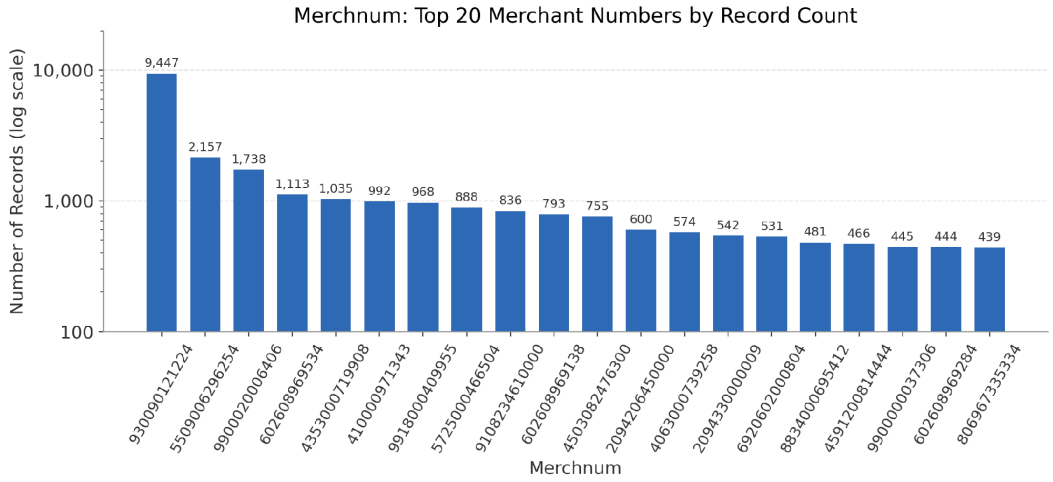


Figure A.4. Top 20 merchant numbers (log scale).

Field: Merch description

Description: Text description (name) of the merchant. The field is fully populated with 13,126 unique values, slightly more than the number of merchant numbers because some merchants appear under several descriptions (for example, FedEx descriptions include the shipping date). The chart shows the 20 most common descriptions. The most common is GSA-FSS-ADV with 1,711 records. Many of the top merchants are office, laboratory and computer suppliers, consistent with government business purchases.

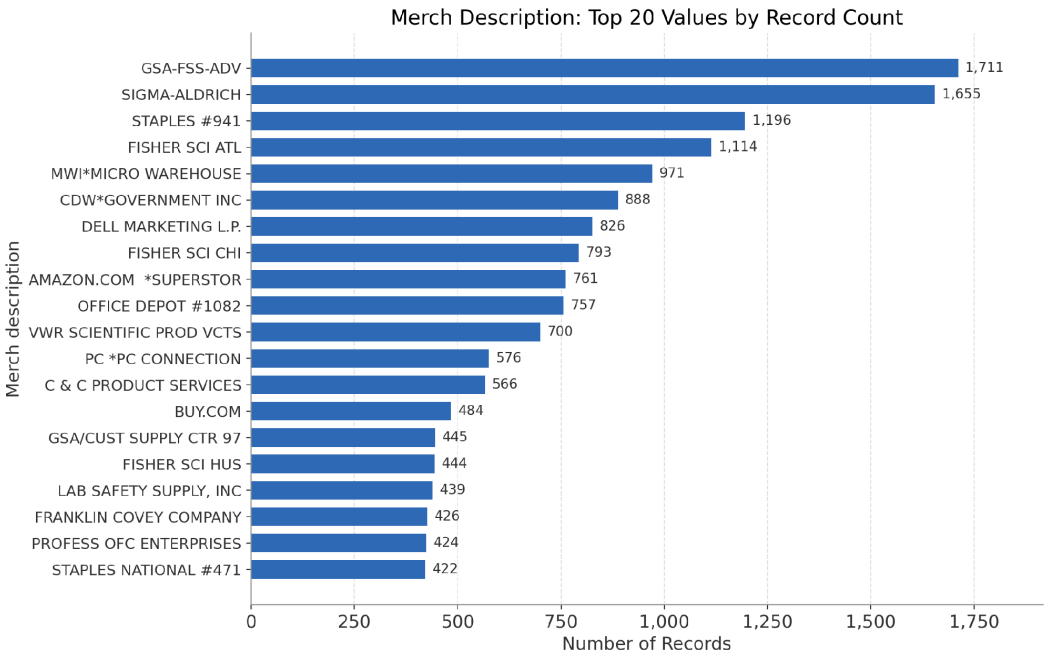


Figure A.5. Top 20 merchant descriptions.

Field: Merch state

Description: Two-letter state code of the merchant. The field is 98.77% populated with 227 unique values. 51 of these are the 50 U.S. states plus DC. The remaining 176 values are non-U.S. codes: Canadian province codes such as ON, BC and PQ (about 200 records) and 168 different three-digit numeric codes (186 records), which appear to be data entry errors. The chart shows the top 20 states. Tennessee (TN) is the most common with 12,206 records, followed by Virginia (7,982) and California (6,919).

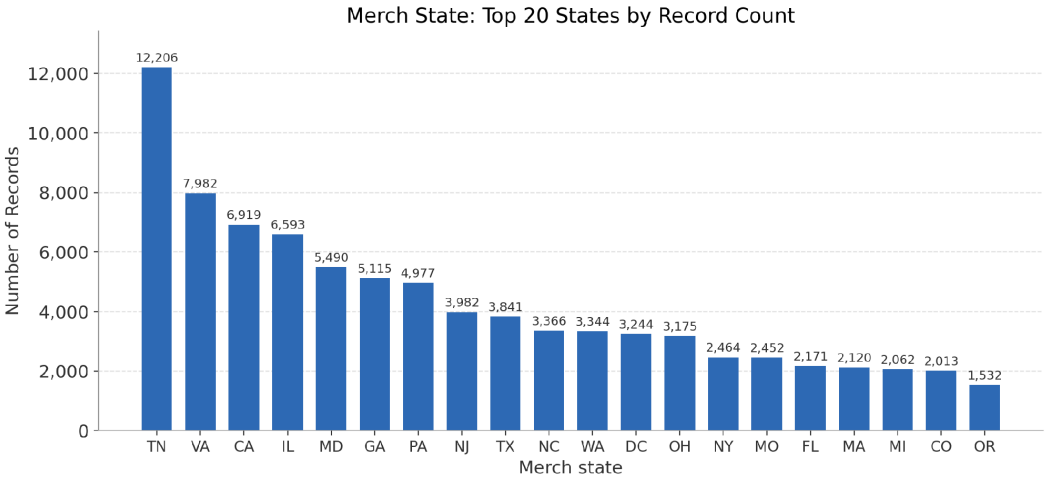


Figure A.6. Top 20 merchant states.

Field: Merch zip

Description: Five-digit zip code of the merchant. The field is 95.19% populated with 4,567 unique values. The zip codes are stored as numbers in the file, so leading zeros have been lost for 8,247

records (for example, 8701 should be 08701); the chart shows them with leading zeros restored. The most common zip code, 38118 (Memphis, TN), has 12,035 records, almost all of which are FedEx charges. The chart shows the top 20 zip codes on a log scale.

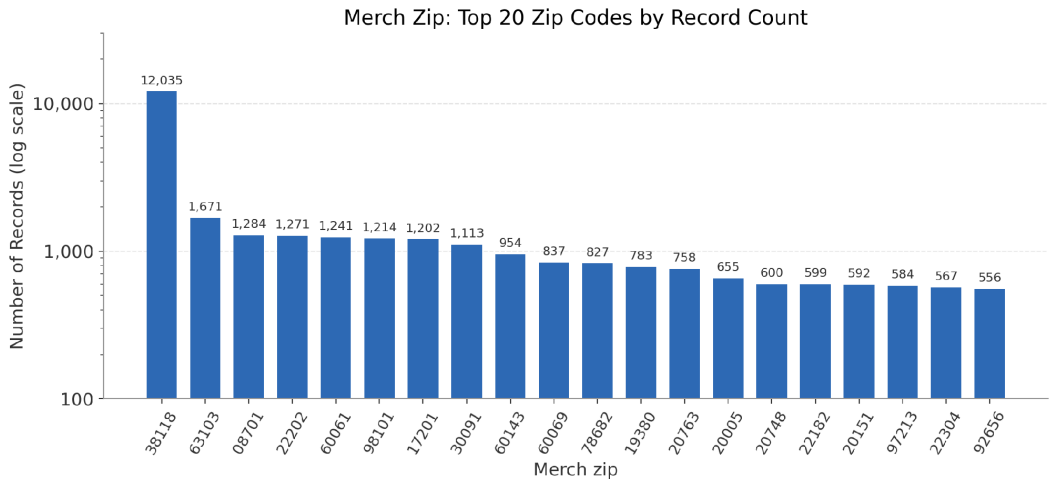


Figure A.7. Top 20 merchant zip codes (log scale).

Field: Transtype

Description: Transaction type code. There are four values. Type P accounts for 97,806 records (99.64%); types A (181), D (173) and Y (1) are rare. The meaning of the codes is not documented; P most likely means purchase. None of the non-P records are labeled as fraud. A log scale is used so the rare types are visible.

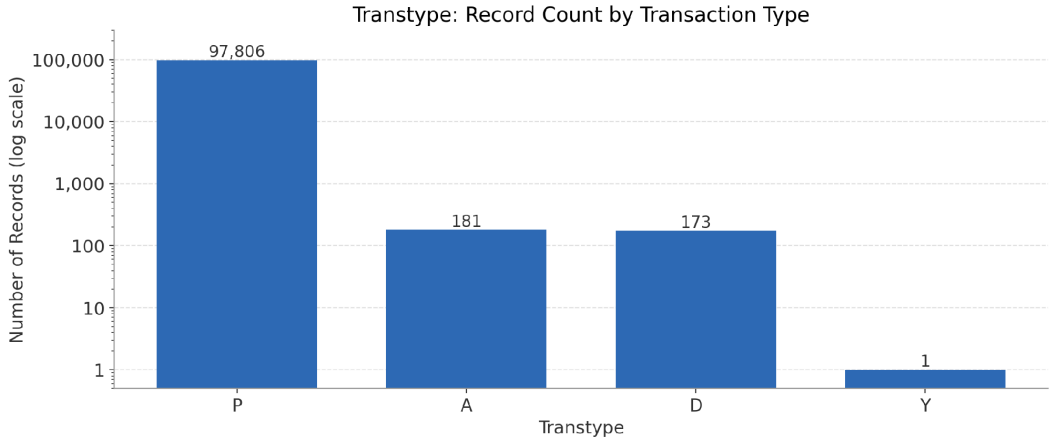


Figure A.8. Transaction types (log scale).

Field: Amount

Description: Dollar amount of the transaction. Amounts range from $0.01 to $3,102,045.53, with a median of $137.20 and a mean of $424.82. The distribution is highly right-skewed. The first chart shows amounts from $0 to $5,000 on a log scale; this covers 99.67% of records, and the 320 records at $5,000 or more are omitted. The count declines steadily with amount but then rises toward $2,500 and falls by more than an order of magnitude just above $2,500, which suggests a single-purchase limit of $2,500 on these cards. The largest amount, $3.1 million, is far above the

next largest ($47,900) and is a clear outlier. The second chart zooms in on $0 to $200 (58.4% of records) and shows a large spike at $3.62, the most common amount (4,283 records), almost all of which are FedEx shipping charges.

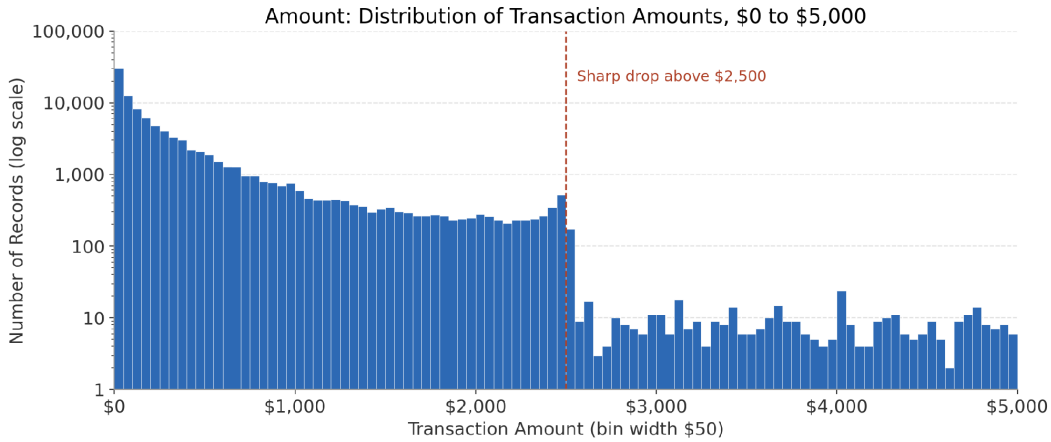


Figure A.9. Amount, $0 to $5,000 (log scale).

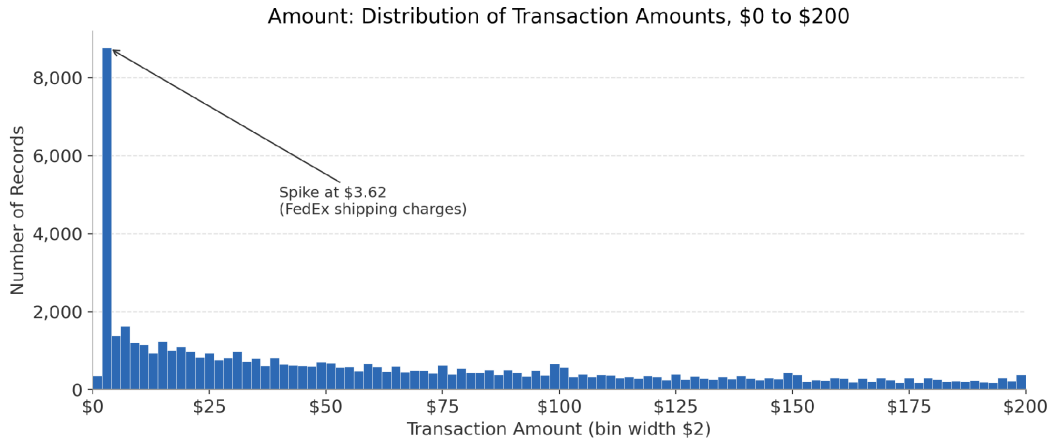


Figure A.10. Amount, $0 to $200.

Field: Fraud

Description: Fraud label, the dependent variable. Fraud = 0 means the transaction is not fraud and Fraud = 1 means it is a fraud. There are 95,878 non-fraud records and 2,283 fraud records, an overall fraud rate of 2.33%.

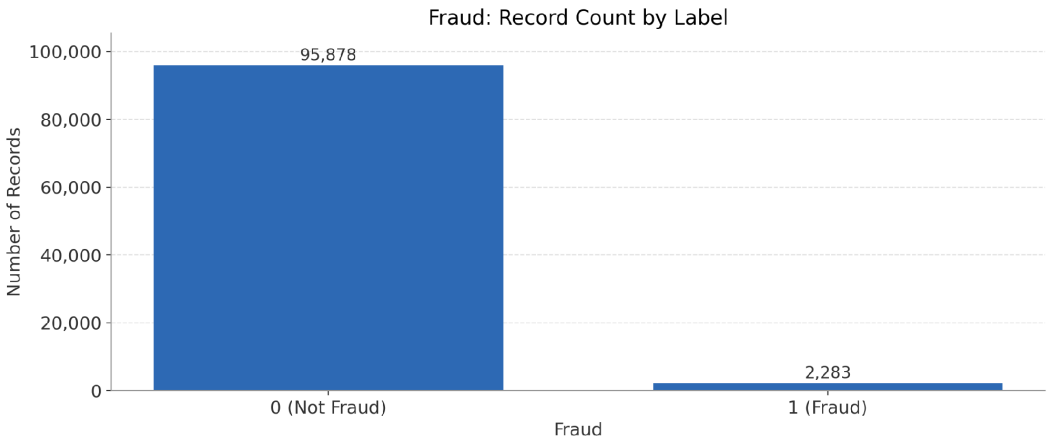


Figure A.11. Fraud label counts.